



南方科技大学

LAB 7: HuggingFace & LLM Evaluation

COE 201 - AI Large Models, Spring 2026

2026-05-17

College of Engineering, SUSTech

Office Hour: Thursday, 14:30-15:30 @ 3rd Research Building, Room 610

Outline

- The Hugging Face Ecosystem 1
- Tokenization Deep Dive 6
- Generative Inference 10
- Structuring Prompts 16
- LLM Evaluation 20
- Lab 7 Exercises 26

The Hugging Face Ecosystem

The standard interface for loading, running, and fine-tuning pre-trained language models.

Labs 3-6: From Scratch

- Implementing attention formulas
- Building Transformer blocks

Lab 7: Industrial Scale

- Using billion-parameter models
- The transformers standard API
- High-performance tokenization

What is Hugging Face?



南方科技大学

Hugging Face is the standard platform for loading, running, and fine-tuning pre-trained LLMs.



Hugging Face

The AI community building the future.

Verified Sponsor

61.4k followers NYC + Paris <https://huggingface.co/> @huggingface

Follow

Pinned

 **transformers** Public
Transformers: the model-definition framework for state-of-the-art machine learning models in text, vision, audio, and multimodal models, for both inference and training.
Python 159k 32.8k

 **diffusers** Public
Diffusers: State-of-the-art diffusion models for image, video, and audio generation in PyTorch.
Python 33.3k 6.9k

 **datasets** Public
The largest hub of ready-to-use datasets for AI models with fast, easy-to-use and efficient data manipulation tools
Python 21.4k 3.2k

 **peft** Public
PEFT: State-of-the-art Parameter-Efficient Fine-Tuning.
Python 20.9k 2.2k

 **accelerate** Public
A simple way to launch, train, and use PyTorch models on almost any device and distributed configuration, automatic mixed precision (including fp8), and easy-to-configure FSDP and DeepSpeed support
Python 9.6k 1.3k

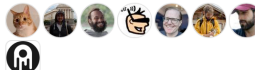
 **optimum** Public
Accelerate inference and training of Transformers, Diffusers, TIMM and Sentence Transformers with easy to use hardware optimization tools
Python 3.4k 633

People



[View all](#)

Sponsoring



Top languages

Python Jupyter Notebook
TypeScript Rust Swift

Recap: The Standard Pipeline



Almost every modern LLM follows the same three-step pipeline:

Text → **Tokenize** → Tensors →
Model Forward → Logits → **Decode** → Text

```
1  from transformers import AutoTokenizer, AutoModelForCausalLM
2  import torch
3
4  # 1. Load model & tokenizer
5  name = "Qwen/Qwen2.5-0.5B"
6  tokenizer = AutoTokenizer.from_pretrained(name)
7  model = AutoModelForCausalLM.from_pretrained(name)
8
9  # 2. Tokenize input text -> Tensors
10 inputs = tokenizer("The capital of France is", return_tensors="pt")
11
```

Python

Recap: The Standard Pipeline



南方科技大学

```
12 # 3. Model Forward -> Logits
13 with torch.no_grad():
14     logits = model(**inputs).logits      # shape: (batch, seq_len, vocab_size)
15     next_token_id = logits[0, -1, :].argmax(dim=-1)
16
17 # 4. Decode -> Text
18 print(tokenizer.decode(next_token_id))  # Output: "Paris"
```

Tokenization Deep Dive

How text becomes numbers the model can process.

Recap: Subword Tokenization (BPE)



Modern models don't split by words or characters. They use **subword** tokenization like Byte-Pair Encoding.

Word-Level (Too Sparse)

- “unbelievable” = 1 token
- “unbelievably” = completely different

Problem: Huge vocabulary.

Method	Tokens	Count
Word-level	“unbelievable”	1
Subword-level	un believ able	3

Subword-Level (Balanced)

- “unbelievable” -
> ["un", "believ", "able"]
- “unbelievably” -
> ["un", "believ", "ably"]

Benefit: Small vocab (30k-150k), handles unseen words.

```
1 tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-0.5B")
2
3 # Encode text -> IDs
4 token_ids = tokenizer.encode("Hello, world!")
5 print(token_ids) # [9707, 11, 1879, 0]
6
7 # Decode back -> text
8 text = tokenizer.decode([9707, 11, 1879])
9 print(text)      # "Hello, world"
```

 Python

Special Tokens: Models reserve certain IDs for structural purposes.

[PAD] (0)

Pad short sequences

[BOS] (1)

Beginning of text

[EOS] (2)

End of generation

Recap: Batching & Attention Masks



Multiple sentences must form a **rectangular tensor** for GPU processing.

```
1 batch = tokenizer([
2     "Hi!",
3     "How are you doing today?"
4 ], padding=True, return_tensors="pt")
5
6 print(batch["input_ids"])
7 # [[ 6783,   0,   0,   0,   0,   0],
8 #   [ 2341, 125, 345, 124, 764,  31]]
9
10 print(batch["attention_mask"])
11 # [[1, 0, 0, 0, 0, 0],
12 #   [1, 1, 1, 1, 1, 1]]
```

Python

Why `attention_mask`?

Without it, Self-Attention would compute weights over `[PAD]` tokens, distorting the meaning of “Hi!”.

The mask (0s) forces attention scores to `-inf` (becomes 0 after softmax).

Generative Inference

How language models actually “talk”.

Recap: Autoregressive Generation

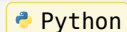


Language models predict the next token x_{t+1} based on context $x_{1...t}$.

Process: 1. **Forward** to compute logits → 2. **Sample** next token → 3. **Update** context → 4. **Repeat** loop until [EOS] .

Task 1 Code Logic:

```
1 while token != EOS:
2     logits = model(input_ids)
3     # Focus on the LAST token's logits
4     next_id = sample(logits[... , -1, :])
5     input_ids += next_id
```



Generation Example:

- **Input:** “The capital of France is”
- **Predict:** “Paris”
- **New Input:** “... is Paris”
- **Predict:** “, the”
- **LOOPING ...**
- **End:** Until [EOS] generated.

How do we pick the next token from the logits?

Greedy Search

Always pick the highest probability token (`argmax`).

- Good for facts and math
- Bad for creativity; repetitive loops

Random Sampling

Roll a dice weighted by probabilities (`torch.multinomial`).

- Outputs feel natural and varied
- May occasionally select nonsensical tokens

Temperature (T) controls sampling:

$$P(x_i) = \frac{\exp\left(\frac{l_i}{T}\right)}{\sum_j \exp\left(\frac{l_j}{T}\right)}$$

Temperature effect:

$T \rightarrow 0$: sharp distribution, nearly greedy

$T = 1$: original distribution

$T \rightarrow \infty$: uniform distribution, pure random

To prevent sampling from the “long tail” of unlikely tokens, we truncate the distribution

Nuclear Set $V^{(p)} \subset V$: *Smallest set of tokens whose cumulative probability exceeds threshold p .*

$$\sum_{x \in V^{(p)}} P(x \mid x_{<t}) \geq p$$

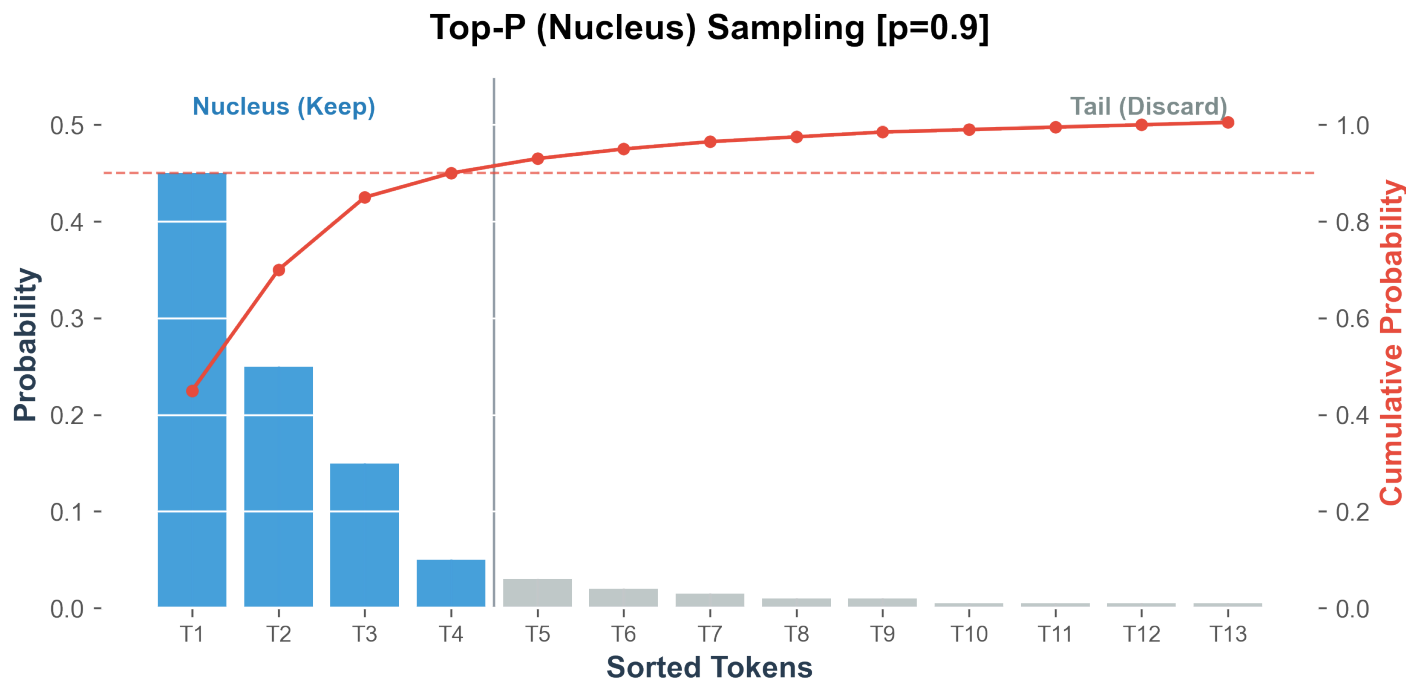
Top-P Logic

1. Sort tokens descending by prob.
2. Keep until cumulative sum $\geq p$.
3. Set logits of remaining to $-\infty$.
4. Sample from surviving tokens.

Adaptive Truncation:

- **Confident model**: Top token = 95% $\rightarrow V^p$ contains only 1 candidate.
- **Uncertain model**: Top token = 10% $\rightarrow V^p$ includes many candidates.

Top-P (Nucleus) Sampling



- Sort tokens by probability then cumulative sum $\geq p$
- Keep the tokens in the nucleus, then do sampling

Decoding in Hugging Face API



`transformers` provides a built-in `generate()` method:

```
1  outputs = model.generate(  
2      input_ids,  
3      max_new_tokens=50,  
4      # Greedy Search (default)  
5      # do_sample=False  
6      # Random Sampling with Temp & Top-P  
7      do_sample=True,  
8      temperature=0.8,  
9      top_p=0.9,  
10     # Optional safeguards  
11     repetition_penalty=1.2,  
12     pad_token_id=tokenizer.eos_token_id  
13 )  
14 print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

 Python

Structuring Prompts

How instruct-tuned models expect to be spoken to.

Base Models (Pre-trained)

- **Behavior:** Next-token prediction.
- **Data:** Internet-level raw text (billion-level tokens).

Prompt: “What is the capital of France?”

Response: “What is the capital of China? What is the capital of...” (✗ Just completing)

Instruct Models (Post-trained)

- **Behavior:**
 - Answer in dialogue format (Chat Templates);
 - Follow complex user instructions;
 - Refuse harmful requests (**Safety**).
- **Data:** (Prompt, Answer) pairs.

Prompt: “What is the capital of France?”

Response: “The capital of France is Paris.” (✓
Direct answer)

Instruct models expect a strict format separating “who is speaking”:

- **System**: High-level behavior (“You are a helpful assistant.”)
- **User**: The user’s query
- **Assistant**: The model’s response

Different models use different formats! e.g. ChatML (Qwen, Mistral, etc.) uses special control tokens to mark:

```
1 <|im_start|>user
2 Hi there!<|im_end|>
3 <|im_start|>assistant
4 Nice to meet you!<|im_end|>
5 <|im_start|>user
6 Can I ask a question?<|im_end|>
```

 Text

Hugging Face handles it automatically:

```
1  messages = [  
2      {"role": "system", "content": "You are a helpful assistant."},  
3      {"role": "user", "content": "Tell me a joke."},  
4  ]  
5  # Automatically uses the correct format for this model  
6  formatted = tokenizer.apply_chat_template(  
7      messages,  
8      tokenize=False,  
9      add_generation_prompt=True # Appends  
10 )  
11 #For Qwen: "<|im_start|>system\nYou are a helpful assistant.<|im_end|>\n<|im_start|>user\nTell me a joke.<|im_end|>\n<|im_start|>assistant\n"
```

Python

Key benefit: One API works for all models (Qwen, Llama, Deepseek, etc.). Each model's ChatTemplate format is specified in its configuration.

LLM Evaluation

Measuring the performance and reliability of language models.

How do we know if a model is actually “good”?

Closed-ended (选择题 MCQ)

- **Nature:** Fixed options & deterministic answers.
- **Tasks:** MMLU (MCQ), GSM8K (Math)
- **Metric:** Accuracy / Exact Match (EM)

Open-ended (问答题 FRQ)

- **Nature:** Free-form content.
- **Tasks:** Dialogue, Summarization
- **Metric:** ROUGE / BLEU / LLM-Judge

Manual vs Automatic: Deterministic metrics are fast but struggle with “semantic” correctness. (See **LLM-as-a-judge** next).

Accuracy & Exact Match (EM)

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{y}_i = y_i)$$

- **Logic:** Direct string-level or label matching.
- **Best for:** Deterministic tasks (Math, MCQ).

BLEU (Precision-based)

$$p_n = \frac{\text{Count}_{\text{match}}(\text{n-gram})}{\text{Count}_{\text{gen}}(\text{n-gram})}$$

- **Logic:** Percentage of correct words in output.
- **Focus: Precision** (Avoids noise).

ROUGE (Recall-based)

$$\text{ROUGE-N} = \frac{\text{Count}_{\text{match}}(\text{n-gram})}{\text{Count}_{\text{ref}}(\text{n-gram})}$$

- **Logic:** Percentage of reference words captured.
- **Focus: Recall** (Covers key points).

Key Distinction Summary

- **BLEU:** “Is the generated text correct?”
- **ROUGE:** “Is the generated text complete?”

Both rely on n -gram overlap statistics.

Beyond comparing outputs, we can measure how well the model **predicts** text.

Perplexity (PPL)

$$\text{PPL} = \exp \left(\frac{1}{N} \sum_i -\log p(x_i \mid x_{<i}) \right)$$

Average “surprise” the model has when predicting each token.

Interpretation:

- **Low PPL:** Model finds the text predictable (fits training distribution)
- **High PPL:** Model is “surprised” by the text (unfamiliar style/content)
- Compare model quality, detect dataset drift

Key insight: Perplexity uses the logits you computed in Task 1!

Arena Style: Win Rate & Elo Rating



南方科技大学

Pairwise Comparison (A/B Test)

- Users (or Judges) compare two model outputs side-by-side.
- **Label**: “A is better”, “B is better”.
- **Win Rate**: % of victories against a specific baseline model (e.g., GPT-4).

Elo Rating System (From Chess Ranking)

Relative rating where scores update after “match”:

$$R'_A = R_A + K(S_A - E_A)$$

- E_A : Expected win prob based on rating difference.
- S_A : Actual outcome (1: win, 0.5: tie, 0: loss).



Explore <https://arena.ai/leaderboard/text>

Traditional metrics struggle with “semantic” correctness.

The Problem: Semantic Gap

- **Reference:** “The apple is red.”
- **Wrong Answer:** “The apple is blue.” (ROUGE-1: 0.5)
- **Output:** “The fruit is crimson.” (ROUGE-1: 0.25)
- **Metric Score:** Low overlap → **Actual Quality:** High

The Solution: LLM-as-a-judge

Use a stronger model (e.g., GPT-5.4, Claude Opus 4.6) to evaluate.

- **Logic:** Model understands synonyms and intent beyond n-gram overlap.
- “On a scale of 1-10, how helpful is this response?” ➡ “The fruit is crimson” (10/10)

Lab 7 Exercises

Get hands-on with Hugging Face abstractions.

Task 0: LMArena Judgement [0 points]

- Go to <https://arena.ai/>, choose two models to compare.
- Generate a response from each model to the same prompt.
- Choose the better response and submit your judgement.

Task 1: Model Inference Pipeline (`lab7_model_inference.py`) [30 points]

- Load the `models/Qwen2.5-0.5B` with `transformers` library.
- Implement a manual generation loop: logits -> sample -> append -> repeat.
- Apply temperature scaling and use the provided Top-P filtering helper.

Task 2: Interactive Chatbot (`lab7_hf_chatbot.py`) [30 points]

- Build a CLI chatbot using `model.generate()` with the same model.
- Use `apply_chat_template` to manage conversation history.

Task 3: LLM Evaluation Pipeline (`lab7_metrics.py`) [40 points]

- Implement perplexity calculation using model logits.
- Evaluate the model on a 10-question mini-benchmark using log-likelihood.
- Report perplexity and accuracy metrics.

Happy Coding!